논리회로

Chapter 03 디지털 코드

학습목표

- 다양한 디지털 코드를 구분하여 이해할 수 있다.
- 문자와 숫자를 나타내는 코드를 이해할 수 있다.
- 가중치 코드와 비가중치 코드를 이해하고 이를 활용할 수 있다.
- 에러 검출 코드를 이해하고 이를 활용할 수 있다.

01. BCD 코드와 3초과 코드
02. 다양한 2진 코드들
03. 그레이 코드
04. 에러 검출 코드
05. 영숫자 코드

01. BCD 코드와 3초과 코드

❑ BCD 코드(Binary Coded Decimal Code : 2진화 10진 코드, 8421 코드)

- BCD코드는 10진수 0(0000)부터 9(1001)까지를 2진화한 코드
- 표기는 2진수이지만 의미는 10진수
- 1010부터 1111까지 6개는 사용하지 않음

10진수	BCD 코드
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

10진수	BCD 코드
10	0001 0000
11	0001 0001
12	0001 0010
13	0001 0011
14	0001 0100
15	0001 0101
16	0001 0110
17	0001 0111
18	0001 1000
19	0001 1001

10진수	BCD 코드
20	0010 0000
31	0011 0001
42	0100 0010
53	0101 0011
64	0110 0100
75	0111 0101
86	1000 0110
97	1001 0111
196	0001 1001 0110
237	0010 0011 0111

01. BCD 코드와 3초과 코드

❖ BCD 코드의 연산

10진 덧셈	BCD 덧셈
6	$0110_{(BCD)}$
+ 3	$0011_{(BCD)}$
9	$1001_{(BCD)}$

10진 덧셈	BCD 덧셈
42	$0100\ 0010_{(BCD)}$
+ 37	$0011\ 0111_{(BCD)}$
79	$0111\ 1001_{(BCD)}$

- 계산 결과가 BCD코드를 벗어날 때, 즉, 9(1001)를 초과하는 경우 계산 결과에 6(0110)을 더해준다.

10진 덧셈	BCD 덧셈
8	1000
+ 7	+ 0111
	1111
	+ 0110
15	0001 0101

예제 3-1 69+85를 BCD로 바꾸어 연산한 결과는?

풀이 69와 85를 BCD로 바꾸어 계산하면 1110 1110이 되어 10의 자리와 1의 자리 모두가 9를 초과하므로 계산 결과에 0110 0110(=$66_{(10)}$)를 더해주면 올바른 결과를 얻는다.

10진 덧셈	BCD 덧셈
69	0110 1001
+ 85	+ 1000 0101
	1110 1110
	+ 0110 0110
154	0001 0101 0100

End of Example

❑ 3초과 코드

- BCD코드(8421코드)로 표현된 값에 3을 더해 준 값으로 나타내는 코드
- 자기 보수의 성질을 갖는다.

10진수	BCD 코드	3초과 코드
0	0000	0011
1	0001	0100
2	0010	0101
3	0011	0110
4	0100	0111
5	0101	1000
6	0110	1001
7	0111	1010
8	1000	1011
9	1001	1100

+0011
(3)

자기 보수의 관계

예제 3-2 10진수 13을 3초과 코드로 변환하여라.

풀이 $13_{(10)}$은

- 10의 자리 $0001_{(2)}$ → 3초과 코드 0100
- 1의 자리 $0011_{(2)}$ → 3초과 코드 0110
- 따라서 $13_{(10)} = 0001\ 0011_{(\text{BCD code})} = 0100\ 0110_{(\text{excess 3 code})}$

End of Example

❑ 가중치 코드(weighted code)

- 그 위치에 따라 정해진 값을 갖는 코드

10진수	8421 코드 (BCD)	2421 코드	5421 코드	84-2-1 코드	51111 코드	바이퀴너리코드 (Biquinary Code) 5043210	링 카운터 (ring counter) 9876543210
0	0000	0000	0000	0000	00000	0100001	0000000001
1	0001	0001	0001	0111	00001	0100010	0000000010
2	0010	0010	0010	0110	00011	0100100	0000000100
3	0011	0011	0011	0101	00111	0101000	0000001000
4	0100	0100	0100	0100	01111	0110000	0000010000
5	0101	1011	1000	1011	10000	1000001	0000100000
6	0110	1100	1001	1010	11000	1000010	0001000000
7	0111	1101	1010	1001	11100	1000100	0010000000
8	1000	1110	1011	1000	11110	1001000	0100000000
9	1001	1111	1100	1111	11111	1010000	1000000000

❖ 8421 코드(BCD 코드)

10진수	8421코드	10진수 계산
0	0000	8×0 + 4×0 + 2×0 + 1×0 = 0
1	0001	8×0 + 4×0 + 2×0 + 1×1 = 1
2	0010	8×0 + 4×0 + 2×1 + 1×0 = 2
3	0011	8×0 + 4×0 + 2×1 + 1×1 = 3
4	0100	8×0 + 4×1 + 2×0 + 1×0 = 4
5	0101	8×0 + 4×1 + 2×0 + 1×1 = 5
6	0110	8×0 + 4×1 + 2×1 + 1×0 = 6
7	0111	8×0 + 4×1 + 2×1 + 1×1 = 7
8	1000	8×1 + 4×0 + 2×0 + 1×0 = 8
9	1001	8×1 + 4×0 + 2×0 + 1×1 = 9

☞ 자기보수 성질 없음

❖ 2421 코드

10진수	2421 코드	10진수 계산	2421 코드	10진수 계산
0	0000	2×0+4×0+2×0+1×0=0	0000	2×0+4×0+2×0+1×0=0
1	0001	2×0+4×0+2×0+1×1=1	0001	2×0+4×0+2×0+1×1=1
2	0010	2×0+4×0+2×1+1×0=2	1000	2×1+4×0+2×0+1×0=2
3	0011	2×0+4×0+2×1+1×1=3	1001	2×1+4×0+2×0+1×1=3
4	0100	2×0+4×1+2×0+1×0=4	1010	2×1+4×0+2×1+1×0=4
5	1011	2×1+4×0+2×1+1×1=5	0101	2×0+4×1+2×0+1×1=5
6	1100	2×1+4×1+2×0+1×0=6	0110	2×0+4×1+2×1+1×0=6
7	1101	2×1+4×1+2×0+1×1=7	0111	2×0+4×1+2×1+1×1=7
8	1110	2×1+4×1+2×1+1×0=8	1110	2×1+4×1+2×1+1×0=8
9	1111	2×1+4×1+2×1+1×1=9	1111	2×1+4×1+2×1+1×1=9

☞ 자기보수 성질을 가짐

❖ 5421 코드

10진수	5421 코드	10진수 계산	5421 코드	10진수 계산
0	0000	5×0+4×0+2×0+1×0=0	0000	5×0+4×0+2×0+1×0=0
1	0001	5×0+4×0+2×0+1×1=1	0001	5×0+4×0+2×0+1×1=1
2	0010	5×0+4×0+2×1+1×0=2	0010	5×0+4×0+2×1+1×0=2
3	0011	5×0+4×0+2×1+1×1=3	0011	5×0+4×0+2×1+1×1=3
4	0100	5×0+4×1+2×0+1×0=4	0100	5×0+4×1+2×0+1×0=4
5	1000	5×1+4×0+2×0+1×0=5	0101	5×0+4×1+2×0+1×1=5
6	1001	5×1+4×0+2×0+1×1=6	0110	5×0+4×1+2×1+1×0=6
7	1010	5×1+4×0+2×1+1×0=7	0111	5×0+4×1+2×1+1×1=7
8	1011	5×1+4×0+2×1+1×1=8	1011	5×1+4×0+2×1+1×1=8
9	1100	5×1+4×1+2×0+1×0=9	1100	5×1+4×1+2×0+1×0=9

☞ 자기보수 성질 없음

❖ 84-2-1 코드($84\overline{2}\overline{1}$ 코드)

10진수	84-2-1코드	10진수 계산
0	0000	8×0+4×0-2×0-1×0=0
1	0111	8×0+4×1-2×1-1×1=1
2	0110	8×0+4×1-2×1-1×0=2
3	0101	8×0+4×1-2×0-1×1=3
4	0100	8×0+4×1-2×0-1×0=4
5	1011	8×1+4×0-2×1-1×1=5
6	1010	8×1+4×0-2×1-1×0=6
7	1001	8×1+4×0-2×0-1×1=7
8	1000	8×1+4×0-2×0-1×0=8
9	1111	8×1+4×1-2×1-1×1=9

☞ 자기보수 성질을 가짐

❑ 비가중치 코드(non-weighted code)

- 각각의 위치에 해당하는 값이 없는 코드
- 데이터 변환과 같은 특수한 용도로 사용되기 위한 코드 (2-out-of-5)

10진수	3초과 코드	5중 2코드 (2-out-of-5)	shift counter	그레이 코드
0	0011	11000	00000	0000
1	0100	00011	00001	0001
2	0101	00101	00011	0011
3	0110	00110	00111	0010
4	0111	01001	01111	0110
5	1000	01010	11111	0111
6	1001	01100	11110	0101
7	1010	10001	11100	0100
8	1011	10010	11000	1100
9	1100	10100	10000	1101

예제 3-3 10진수 3864를 2421코드로 변환하시오.

풀이 각 자리 별로 변환하면 다음과 같다. 여기서 3, 6, 4는 각각 2가지 경우가 존재한다.

3	8	6	4
0011 or 1001	1110	1100 or 0110	0100 or 1010

End of Example

예제 3-4

74-2-1코드로 표현한 아래의 수를 10진수로 나타내어라.
(a) 0110 (b) 1100 (c) 1001 (d) 1011

풀이

(a) $0110 = 7 \times 0 + 4 \times 1 + (-2) \times 1 + (-1) \times 0 = 4-2 = 2$

(b) $1100 = 7 \times 1 + 4 \times 1 + (-2) \times 0 + (-1) \times 0 = 7+4 = 11$

(c) $1001 = 7 \times 1 + 4 \times 0 + (-2) \times 0 + (-1) \times 1 = 7-1 = 6$

(d) $1011 = 7 \times 1 + 4 \times 0 + (-2) \times 1 + (-1) \times 1 = 7-2-1 = 4$

End of Example

❑ 그레이 코드(Gray Code)

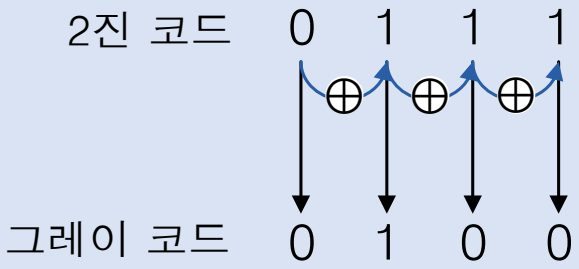
- 가중치가 없는 코드이기 때문에 연산에는 부적당하지만, 아날로그-디지털 변환기나 입출력 장치 코드로 주로 쓰인다.
- 연속되는 코드들 간에 하나의 비트만 변화하여 새로운 코드가 된다.

10진수	2진 코드	그레이 코드
0	0000	0000
1	0001	0001
2	0010	0011
3	0011	0010
4	0100	0110
5	0101	0111
6	0110	0101
7	0111	0100

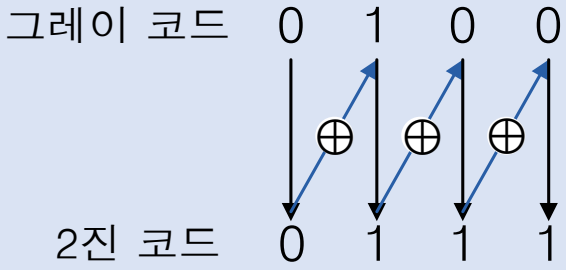
10진수	2진 코드	그레이 코드
8	1000	1100
9	1001	1101
10	1010	1111
11	1011	1110
12	1100	1010
13	1101	1011
14	1110	1001
15	1111	1000

이웃하는 코드 간에 한 비트만 다르다.

2진 코드를 그레이 코드로 변환하는 방법



그레이 코드를 2진 코드로 변환하는 방법

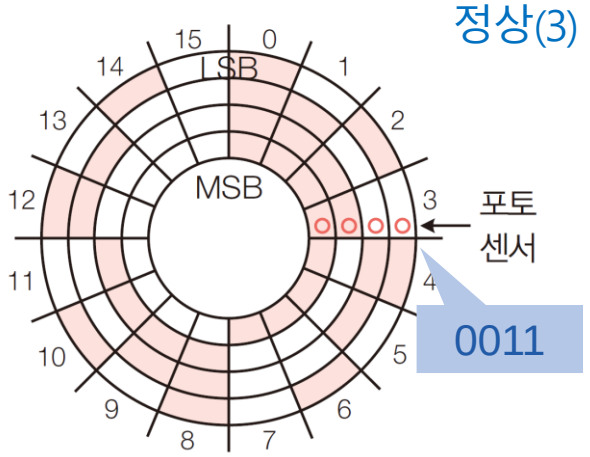
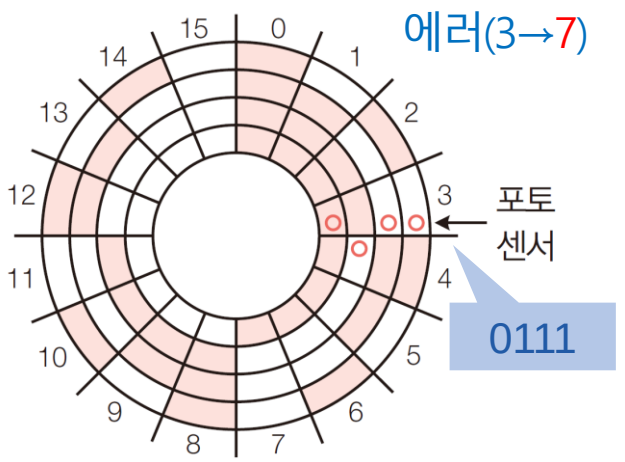
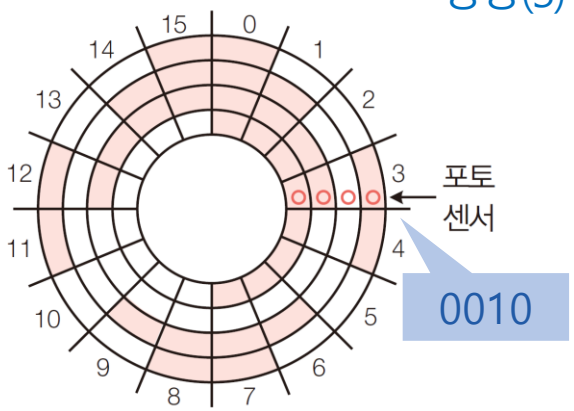
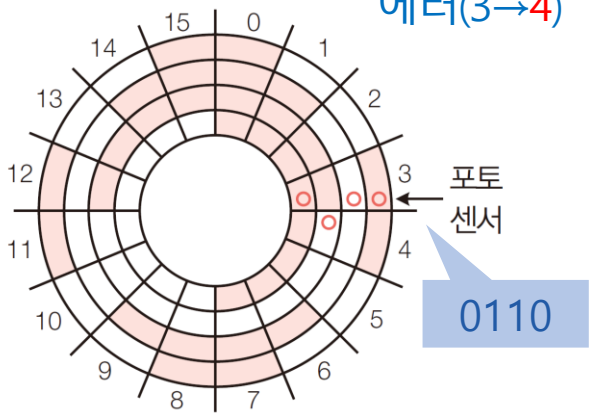


XOR 진리표

입력		출력
A	B	F
0	0	0
0	1	1
1	0	1
1	1	0

$$F = A \oplus B$$

❑ 그레이 코드 입력장치 적용 예

2진코드		
그레이코드		

0

1

그레이 코드는
오차가 적음

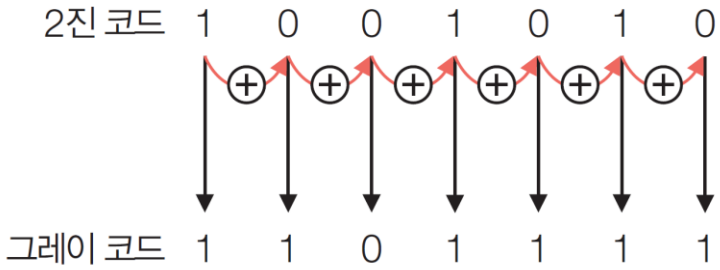
예제 3-5

다음 2진 코드는 그레이 코드로, 그레이 코드는 2진 코드로 변환하시오.

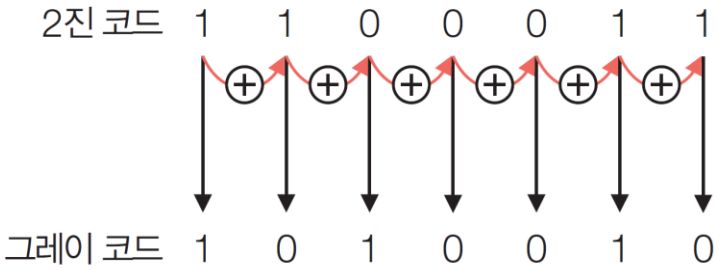
(a) $1001010_{(2)}$ (b) $1100011_{(2)}$ (c) $1001010_{(G)}$ (d) $1011101_{(G)}$

풀이

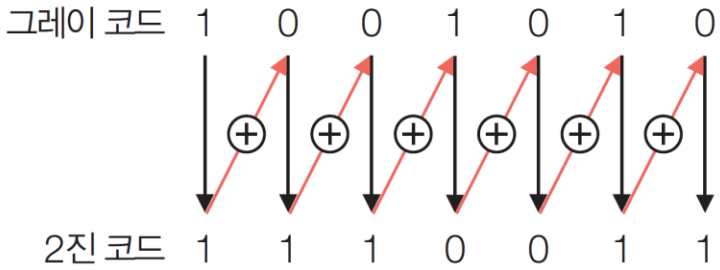
(a) $1001010_{(2)}$



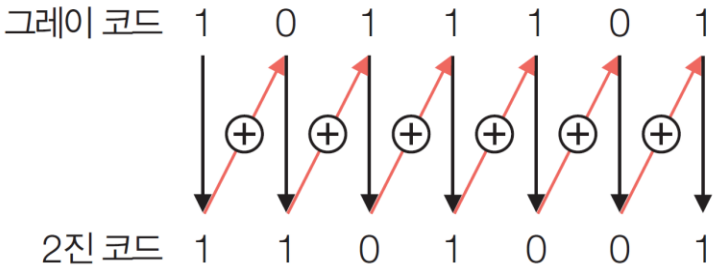
(b) $1100011_{(2)}$



(c) $1001010_{(G)}$



(d) $1011101_{(G)}$



End of Example

1 패리티 비트

- 짝수 패리티(even parity) : 데이터에서 1의 개수를 짝수 개로 맞춤
- 홀수 패리티(odd parity) : 1의 개수를 홀수 개로 맞춤
- 패리티 비트는 데이터 전송과정에서 에러 검사를 위한 추가비트. 패리티는 단지 에러 검출만 가능하며, 여러 비트에 에러가 발생할 경우에는 검출이 안될 수도 있음

7비트 ASCII 코드에 패리티 비트를 추가한 코드

데이터	짝수 패리티	홀수 패리티
…	…	…
A	0 1000001	1 1000001
B	0 1000010	1 1000010
C	1 1000011	0 1000011
D	0 1000100	1 1000100
…	…	…

예제 3-6

짝수 패리티 시스템에서 코드 그룹 10110, 11010, 110011, 10101110100, 1100010101011을 수신하였다. 에러가 발생한 그룹을 찾아보시오.

풀이

- 짝수 패리티가 필요하므로 1이 홀수 개인 그룹은 에러 발생
- 10110, 11010, 1100010101011에 비트 에러가 발생하였음

End of Example

❑ 병렬 패리티(parallel parity)

- 패리티를 블록 데이터에 적용해서 가로와 세로 데이터들에 대해서 패리티를 적용하면 에러를 검출하여 그 위치를 찾아 정정할 수 있다.

1	0	1	0	1	1	1	1	*0*
1	0	0	0	0	0	1	1	*1*
0	1	0	0	0	0	0	0	*1*
1	1	1	1	0	0	0	0	*0*
1	0	1	1	1	0	0	1	*1*
0	0	0	0	0	1	1	1	*1*
1	1	1	1	1	1	1	1	*0*
0	1	1	1	1	0	0	0	*0*
1	*0*	*1*	*0*	*0*	*1*	*0*	*1*	*0*

원래 데이터 블록

가로세로 모두 1의 개수가 짝수임

1	0	1	0	1	1	1	1	*0*
1	0	0	0	0	0	1	1	*1*
0	1	0	0	0	0	0	0	*1*
1	1	1	1	0	0	0	0	*0*
1	0	1	1	*0*	0	0	1	*1*
0	0	0	0	0	1	1	1	*1*
1	1	1	1	1	1	1	1	*0*
0	1	1	1	1	0	0	0	*0*
1	*0*	*1*	*0*	*0*	*1*	*0*	*1*	*0*

에러가 발생한 블록

가로에 1의 개수가 5개임

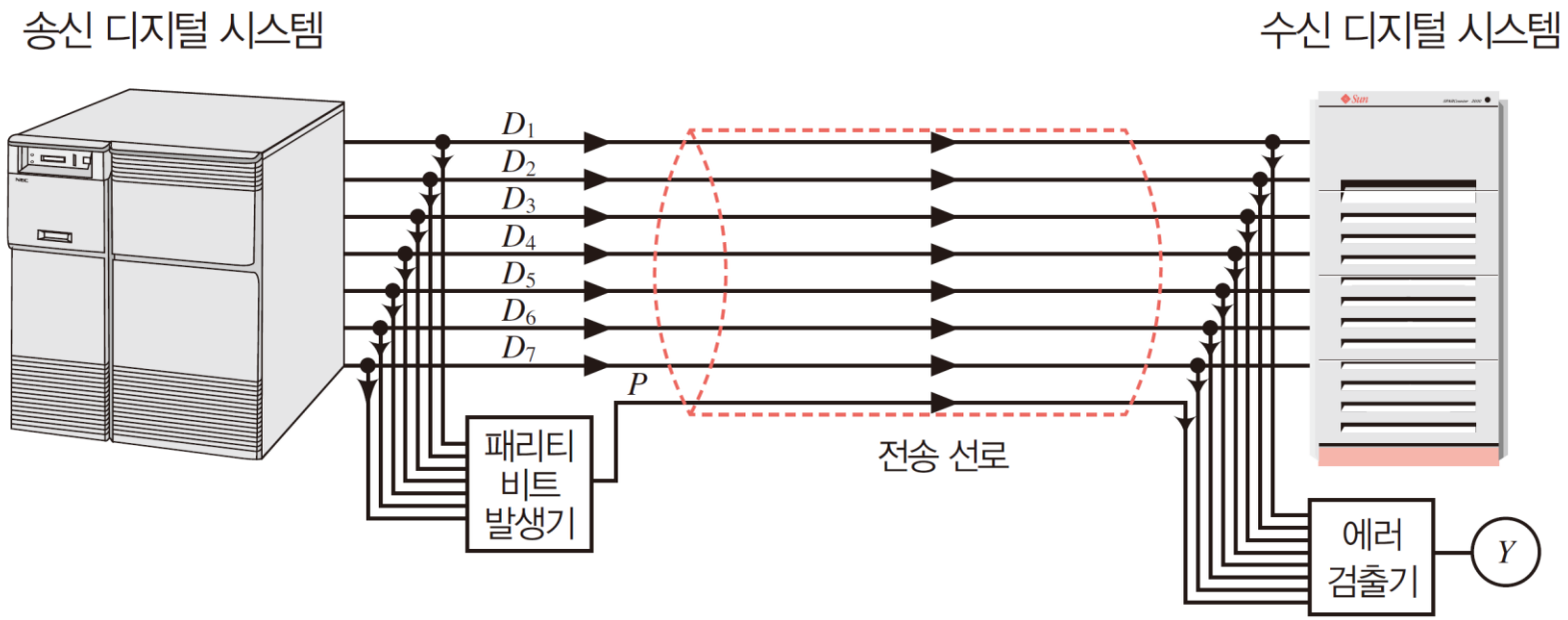
겹치는 부분 에러

세로에 1의 개수가 3개임

❑ 데이터 전송 시스템에서 패리티 비트를 사용한 에러 검출

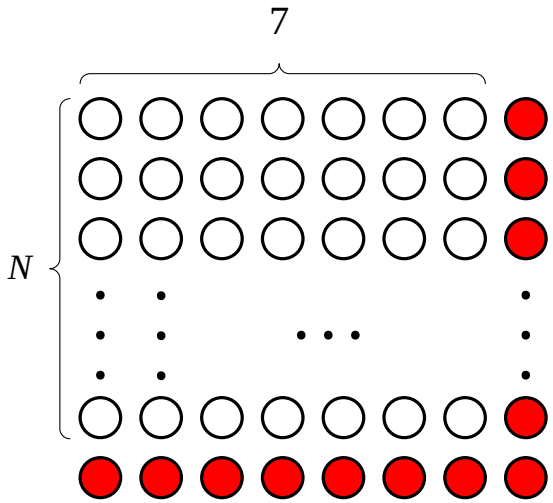
- 에러를 검출하기 위하여 송신측에 패리티 발생기를 구성하고 수신측에는 패리티 검출기를 구성하여 그 출력을 보고 에러 발생 여부를 판단

짝수 패리티	Y=0(에러 없음), Y=1(에러 발생)
홀수 패리티	Y=1(에러 없음), Y=0(에러 발생)



예제 3-7 N개의 ASCII 문자를 병렬 패리티(수평, 수직 패리티) 문자로 검사할 때 코드의 효율을 구하여라.

풀이 ASCII 문자 N개와 패리티 비트를 포함하여 전송하는 총 비트 수는 $8(N+1)$이다. 또한 전송되는 순수 데이터 비트 수는 $7N$ 이다.



$$\text{효율} = \frac{\text{전송 비트 수}}{\text{총비트 수}} = \frac{7N}{8(N+1)}$$

단순한 직렬 패리티 전송인 경우 효율은 87.5%이다.

$$\frac{7N}{8N} = 0.875$$

End of Example

8비트 데이터 +1비트 패리티 로 송신할 때 전송효율은?

2 에러 정정 코드 : 해밍(hamming code)코드

- 에러를 검출도 하고 1비트 오류일 경우에는 수정도 할 수 있는 코드
- 추가적으로 많은 비트가 필요하므로 많은 양의 데이터 전달이 필요
- 데이터 비트와 패리티 비트와의 관계

$$2^{p-1} - p + 1 \le d \le 2^{p} - p - 1 \qquad p \ge 2$$

p는 패리티 비트의 수, d는 데이터 비트의 수,

- $p = 4$일 때, $2^{4-1} - 4 + 1 \le d \le 2^{4} - 4 - 1$이므로 $5 \le d \le 11$이다.
- 따라서 데이터 비트수가 5개 이상 11개 이하일 때 패리티는 4개가 필요하다.
- 패리티 비트의 위치는 앞에서 부터 $2^0, 2^1, 2^2, 2^3, 2^4, \ldots$번째, 즉 $1, 2, 4, 8, 16, \ldots$ 번째이다.
- 데이터 비트는 나머지 위치에 순서대로 들어간다.

- 해밍 코드에서는 짝수 패리티를 사용하여 P1, P2, P4, … 를 구함
  - **P1 은 한 자리씩 건너서 패리티 계산하여 넣음**
  - **P2는 2자리씩 건너서 패리티 계산하여 넣음**
  - **P4는 4자리씩 건너서 패리티 계산하여 넣음**

$$P_1 = D_3 \oplus D_5 \oplus D_7 \oplus D_9 \oplus D_{11}$$
$$P_2 = D_3 \oplus D_6 \oplus D_7 \oplus D_{10} \oplus D_{11}$$
$$P_4 = D_5 \oplus D_6 \oplus D_7 \oplus D_{12}$$
$$P_8 = D_9 \oplus D_{10} \oplus D_{11} \oplus D_{12}$$

비트 위치	1	2	3	4	5	6	7	8	9	10	11	12
변수	P_1	P_2	D_3	P_4	D_5	D_6	D_7	P_8	D_9	D_{10}	D_{11}	D_{12}
P_1 영역	✓		✓		✓		✓		✓		✓	
P_2 영역		✓	✓			✓	✓			✓	✓	
P_4 영역				✓	✓	✓	✓					✓
P_8 영역								✓	✓	✓	✓	✓

❖ 8비트 데이터의 에러 정정 코드

$$P_1 = D_3 \oplus D_5 \oplus D_7 \oplus D_9 \oplus D_{11}$$
$$P_2 = D_3 \oplus D_6 \oplus D_7 \oplus D_{10} \oplus D_{11}$$
$$P_4 = D_5 \oplus D_6 \oplus D_7 \oplus D_{12}$$
$$P_8 = D_9 \oplus D_{10} \oplus D_{11} \oplus D_{12}$$

- for example

P_1	P_2	D_3	P_4	D_5	D_6	D_7	P_8	D_9	D_{10}	D_{11}	D_{12}
		0		0	1	0		1	1	1	0

$$P_1 = D_3 \oplus D_5 \oplus D_7 \oplus D_9 \oplus D_{11} = 0 \oplus 0 \oplus 0 \oplus 1 \oplus 1 = 0$$
$$P_2 = D_3 \oplus D_6 \oplus D_7 \oplus D_{10} \oplus D_{11} = 0 \oplus 1 \oplus 0 \oplus 1 \oplus 1 = 1$$
$$P_4 = D_5 \oplus D_6 \oplus D_7 \oplus D_{12} = 0 \oplus 1 \oplus 0 \oplus 0 = 1$$
$$P_8 = D_9 \oplus D_{10} \oplus D_{11} \oplus D_{12} = 1 \oplus 1 \oplus 1 \oplus 0 = 1$$

❖ 해밍 코드에서 패리티 비트 생성 과정

비트위치	1	2	3	4	5	6	7	8	9	10	11	12
기호	P_1	P_2	D_3	P_4	D_5	D_6	D_7	P_8	D_9	D_{10}	D_{11}	D_{12}
원본 데이터			0		0	1	0		1	1	1	0
P_1 영역	0		0		0		0		1		1	
P_2 영역		1	0			1	0			1	1	
P_4 영역				1	0	1	0					0
P_8 영역								1	1	1	1	0
생성된 코드	0	1	0	1	0	1	0	1	1	1	1	0

생성된 패리티

❖ 해밍 코드에서 패리티 비트 검사 과정

- 전송된 데이터 : 010111011110

P_1	P_2	D_3	P_4	D_5	D_6	D_7	P_8	D_9	D_{10}	D_{11}	D_{12}
0	1	0	1	1	1	0	1	1	1	1	0

☞ 패리티들을 포함하여 검사

$$P_{1c} = P_1 \oplus D_3 \oplus D_5 \oplus D_7 \oplus D_9 \oplus D_{11} = 0 \oplus 0 \oplus 1 \oplus 0 \oplus 1 \oplus 1 = 1$$
$$P_{2c} = P_2 \oplus D_3 \oplus D_6 \oplus D_7 \oplus D_{10} \oplus D_{11} = 1 \oplus 0 \oplus 1 \oplus 0 \oplus 1 \oplus 1 = 0$$
$$P_{4c} = P_4 \oplus D_5 \oplus D_6 \oplus D_7 \oplus D_{12} = 1 \oplus 1 \oplus 1 \oplus 0 \oplus 0 = 1$$
$$P_{8c} = P_8 \oplus D_9 \oplus D_{10} \oplus D_{11} \oplus D_{12} = 1 \oplus 1 \oplus 1 \oplus 1 \oplus 0 = 0$$

- 검사된 패리티를 $P_{8c}\,P_{4c}\,P_{2c}\,P_{1c}$ 순서대로 정렬
- 모든 패리티가 0이면 에러 없음
- 하나라도 1이 있으면 에러 발생: 결과가 0101이므로 에러 있으며 이 결과는 에러 위치를 나타냄
- 0101을 10진수로 바꾸면 5이며, 수신된 데이터에서 앞에서 5번째 비트 0101**1**1011110에 에러가 발생한 것이므로 0101**0**1011110으로 바꾸어 주면 에러가 정정된다. (위치 번호는 1부터 시작)

❖ 해밍 코드에서 에러가 발생한 경우 교정

비트 위치		1	2	3	4	5	6	7	8	9	10	11	12
기호		P_1	P_2	D_3	P_4	D_5	D_6	D_7	P_8	D_9	D_{10}	D_{11}	D_{12}
해밍 코드		0	1	0	1	1	1	0	1	1	1	1	0
P_1 계산	1	0		0		1		0		1		1	
P_2 계산	0		1	0			1	0			1	1	
P_4 계산	1				1	1	1	0					0
P_8 계산	0								1	1	1	1	0

$P_8P_4P_2P_1 = 0101 = \mathbf{5}$: 5번째 비트에 에러 발생, $1 \rightarrow 0$으로 교정

예제 3-8

다음 해밍 코드 중 에러가 있는지 검사하여라.
1 1 1 1 0 1 0 0 1 0 1 0

풀이

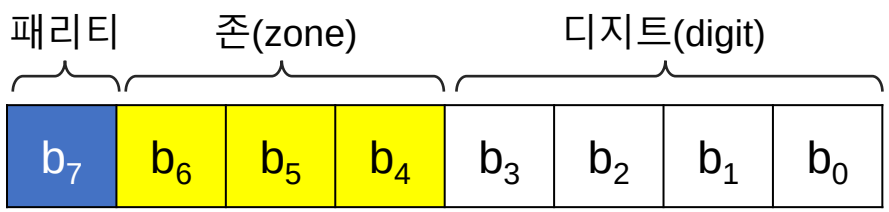
비트 위치		1	2	3	4	5	6	7	8	9	10	11	12
기호		P_1	P_2	D_3	P_4	D_5	D_6	D_7	P_8	D_9	D_{10}	D_{11}	D_{12}
해밍 코드		1	1	1	1	0	1	0	0	1	0	1	0
P_1 계산	0	1		1		0		0		1		1	
P_2 계산	0		1	1			1	0			0	1	
P_4 계산	0				1	0	1	0					0
P_8 계산	0								0	1	0	1	0

$P_8P_4P_2P_1 = 0000$ 이므로 에러 없음

End of Example

1 ASCII (American Standard Code for Information Interchange) 코드

- 미국 국립 표준 연구소(ANSI)가 제정한 정보 교환용 미국 표준 코드
- 128가지의 문자 표현 가능



❑ ASCII 코드의 구성

parity	zone bit			digit bit			
7	6	5	4	3	2	1	0
C	1	0	0	영문자 A~O(0001~1111)			
	1	0	1	영문자 P~Z(0000~1010)			
	0	1	1	숫자 0~9(0000~1001)			

❑ 표준 ASCII 코드표 : 7비트 (비트8는 안 씀)

Digit 4-bit

Zone 3-bit

	0 0000	1 0001	2 0010	3 0011	4 0100	5 0101	6 0110	7 0111	8 1000	9 1001	A 1010	B 1011	C 1100	D 1101	E 1110	F 1111
0 000	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	TAB	LF	VT	FF	CR	SO	SI
1 001	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US
2 010		!	"	#	$	%	&	'	(	)	*	+	,	-	.	/
3 011	0	1	2	3	4	5	6	7	8	9	:	;		=	>	?
4 100	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5 101	P	Q	R	S	T	U	V	W	X	Y	Z	[	₩	]	^	_
6 110	`	a	b	c	d	e	f	g	h	i	j	k	|	m	n	o
7 111	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	DEL

❑ 확장 ASCII 코드표 : 8번째 비트가 1

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
8 1000	Ç	ü	é	â	ä	à	å	ç	ê	ë	è	ï	î	ì	Ä	Å
9 1001	É	æ	Æ	ô	ö	ò	û	ù	ÿ	Ö	Ü	¢	£	¥	Pt	f
A 1010	á	í	ó	ú	ñ	Ñ	ª	º	¿	⌐	¬	½	¼	¡	«	»
B 1011	░	▒	▓	│	┤	╡	╢	╖	╕	╣	║	╗	╝	╜	╛	┐
C 1100	└	┴	┬	├	─	┼	╞	╟	╚	╔	╩	╦	╠	═	╬	╧
D 1101	╨	╤	╥	╙	╘	╒	╓	╫	╪	┘	┌	█	▄	▌	▐	▀
E 1110	α	β	Γ	π	∑	σ	µ	τ	Φ	Θ	Ω	δ	∞	∅	ε	∩
F 1111	≡	±	≥	≤	⌠	⌡	÷	≈	°	∙	·	√	ⁿ	²	■	

예제 3-9

"Digital Logic"의 문자 데이터를 전송하기 위한 1바이트 ASCII 코드를 구하여라. 단, MSB는 짝수 패리티 비트이다.

풀이

D	i	g	i	t	a	l
01000100	01101001	11100111	01101001	01110100	11100001	01101100

(빈칸)	L	o	g	i	c
10100000	11001100	01101111	11100111	01101001	01100011

End of Example

2 표준 BCD 코드

- 6비트로 하나의 문자를 표현 (2비트 zone, 4비트 디지트)
- 최대 64문자까지 표현 가능한 코드

코드의 구성

parity	zone bit		digit bit			
6	5	4	3	2	1	0
C	1	1	영문자 A~I(0001~1001)			
	1	0	영문자 J~R(0001~1001)			
	0	1	영문자 S~Z(0010~1001)			
	0	0	숫자 0~9(0001~1010)			
	혼용		특수문자 및 기타문자			

❑ 표준 BCD 코드표

문자	C ZZ8421	문자	C ZZ8421	문자	C ZZ8421	문자	C ZZ8421	문자	C ZZ8421
A	0 110001	J	1 100001	S	1 010010	1	0 000001	=	0 001011
B	0 110010	K	1 100010	T	0 010011	2	0 000010	>	1 001100
C	1 110011	L	0 100011	U	1 010100	3	1 000011	+	0 010000
D	0 110100	M	1 100100	V	0 010101	4	0 000100	,	1 011011
E	1 110101	N	0 100101	W	0 010110	5	1 000101	)	0 011100
F	1 110110	O	0 100110	X	1 010111	6	1 000110	%	1 011101
G	0 110111	P	1 100111	Y	1 011000	7	0 000111	?	0 011111
H	0 111000	Q	1 101000	Z	0 011001	8	0 001000	-	1 100001
I	1 111001	R	0 101001			9	1 001001	@	1 111010
						0	1 001010	$	1 111111

3 EBCDIC(Extended Binary Coded Decimal Interchange Code) 코드

- 대형 컴퓨터와 IBM 계열 컴퓨터에서 많이 사용되고 있는 8비트 코드(IBM에서 개발)
- 256종류의 문자 코드를 표현할 수 있는 영숫자 코드

코드의 구성

b_9	$b_8\ b_7\ b_6\ b_5$	$b_4\ b_3\ b_2\ b_1$
패리티	존(zone)	디지트(digit)
1	4	4

$b_8\ b_7$		$b_6\ b_5$	
0 0	통신제어문자		
0 1	특수문자		
1 0	소문자	0 0	a ~i
		0 1	j~r
		1 0	s~z
		1 1	
1 1	대문자/숫자	0 0	A~I
		0 1	J~R
		1 0	S~Z
		1 1	0~9

❑ EBCDIC 코드표

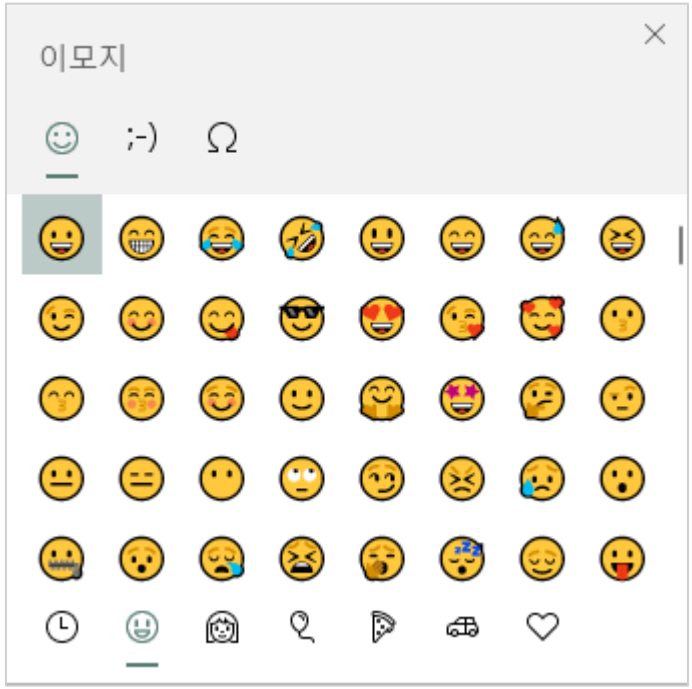
16진		0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
	2진	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
0	0000	NUL	SOH	STX	ETX		HT		DEL				VT	FF	CR	SO	SI
1	0001	DLE						BS		CAN	EM			IFS	IGS	IRS	IUS
2	0010						LF	ETB	ESC						ENQ	ACK	BEL
3	0011			SYN					EOT						NAK		SUB
4	0100	space										[	.		(	+	|
5	0101	&										!	$	*	)		^
6	0110	-	/									|	,	%	_	>	?
7	0111										`	:	#	@	'	=	"
8	1000		a	b	c	d	e	f	g	h	i						
9	1001		j	k	l	m	n	o	p	q	r						
A	1010		~	s	t	u	v	w	x	y	z						
B	1011																
C	1100	{	A	B	C	D	E	F	G	H	I						
D	1101	}	J	K	L	M	N	O	P	Q	R						
E	1110	\		S	T	U	V	W	X	Y	Z						
F	1111	0	1	2	3	4	5	6	7	8	9						

4 유니코드(Unicode)

- 16비트 국제 표준
  - ASCII 코드의 한계성을 극복하기 위하여 개발된 인터넷 시대의 표준으로 2바이트코드로 한 문자를 표시 (65536가지 문자)
  - 2021년 9월 현재 버전 14.0까지 개발되었으며, 모든 문자 인코딩을 유니코드로 통합하는 것이 목적
- 전송포맷
  - 전송할 때에는 유니코드 컨소시엄(IBM, Novell, Microsoft, DEC, Apple 등)에 의해서 32(UTF-32), 16(UTF-16), 8bit(UTF-8)의 세 가지 기본 코드로 변환하여 전송
- 문자 표현 범위
  - 유럽, 중동, 아시아 등 거의 대부분의 문자 포함, 10만 개 이상의 문자로 구성
  - 동남아 및 극동아시아에서 사용하는 한자 70,207자 뿐만 아니라 한글로 표현 가능한 11,172개 문자를 모두 포함
  - 구두표시, 수학기호, 전문기호, 기하학적 모양, 딩벳 기호 등을 포함
- 이모지도 포함
  - 2007년도에 이모지(Emoji)가 도입되기 시작하여, 11.0 버전에 2,784개의 이모지가 사용 중
  - 이모지콘을 입력하려면 윈도우 키와 을 누르면 된다.

이모지콘(참고)

❑ 이모지콘 사용 데이터 입력



- 이모지콘😎은 👍문자이다.
- 🛴를 탈 때는 🧢을 써요.
- 오늘 디저트는 🍧으로 할까?

5 한글 코드

- 한글은 ASCII코드를 기반으로 16비트를 사용하여 하나의 문자를 표현
- 조합형과 완성형으로 분류

조합형

- 조합형으로 표현된 한글은 때에 따라서 다른 응용프로그램에서는 사용할 수 없는 문자들이 많다.
- 조합형은 자음과 모음으로 조합 가능한 모든 한글을 사용할 수 있으며, 심지어 우리나라 고어(古語)까지 취급할 수 있는 장점이 있으나, 출력 시 다시 모아 써야 하는 불편이 있다는 것이 단점이다.

두번째 바이트								첫번째 바이트							
1															
	초성					중성					종성				

완성형

- 1987년 정부가 한국표준으로 정한 것으로 가장 많이 사용되는 한글 음절을 2 바이트의 2진수와 1 대 1로 대응하여 표현하는 방법

한글 유니코드

❑ 한글 전체 코드표

- 한글 코드 범위 (2바이트-16비트) : {AC00-D7AF}
  - 약 11000자의 한글을 2바이트 2진수로 인코딩 (16비트 – 65536)
  - https://jjeong.tistory.com/696

		0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
UTF8: 234, 176, 128;	UNICODE: AC0	가	각	갂	갃	간	갅	갆	갇	갈	갉	갊	갋	갌	갍	갎	갏
UTF8: 234, 176, 144;	UNICODE: AC1	감	갑	값	갓	갔	강	갖	갗	갘	같	갚	갛	개	객	갞	갟
UTF8: 234, 176, 160;	UNICODE: AC2	갠	갡	갢	갣	갤	갥	갦	갧	갨	갩	갪	갫	갬	갭	갮	갯
UTF8: 234, 176, 176;	UNICODE: AC3	갰	갱	갲	갳	갴	갵	갶	갷	갸	갹	갺	갻	갼	갽	갾	갿
UTF8: 234, 177, 128;	UNICODE: AC4	걀	걁	걂	걃	걄	걅	걆	걇	걈	걉	걊	걋	걌	걍	걎	걏
UTF8: 234, 177, 144;	UNICODE: AC5	걐	걑	걒	걓	걔	걕	걖	걗	걘	걙	걚	걛	걜	걝	걞	걟
UTF8: 234, 177, 160;	UNICODE: AC6	걠	걡	걢	걣	걤	걥	걦	걧	걨	걩	걪	걫	걬	걭	걮	걯
UTF8: 234, 177, 176;	UNICODE: AC7	거	걱	걲	걳	건	걵	걶	걷	걸	걹	걺	걻	걼	걽	걾	걿
UTF8: 234, 178, 128;	UNICODE: AC8	검	겁	겂	것	겄	겅	겆	겇	겈	겉	겊	겋	게	겍	겎	겏
UTF8: 234, 178, 144;	UNICODE: AC9	겐	겑	겒	겓	겔	겕	겖	겗	겘	겙	겚	겛	겜	겝	겞	겟
UTF8: 234, 178, 160;	UNICODE: ACA	겠	겡	겢	겣	겤	겥	겦	겧	겨	격	겪	겫	견	겭	겮	겯
UTF8: 234, 178, 176;	UNICODE: ACB	결	겱	겲	겳	겴	겵	겶	겷	겸	겹	겺	겻	겼	경	겾	겿
UTF8: 234, 179, 128;	UNICODE: ACC	곀	곁	곂	곃	계	곅	곆	곇	곈	곉	곊	곋	곌	곍	곎	곏
UTF8: 234, 179, 144;	UNICODE: ACD	곐	곑	곒	곓	곔	곕	곖	곗	곘	곙	곚	곛	곜	곝	곞	곟
UTF8: 234, 179, 160;	UNICODE: ACE	고	곡	곢	곣	곤	곥	곦	곧	골	곩	곪	곫	곬	곭	곮	곯
UTF8: 234, 179, 176;	UNICODE: ACF	곰	곱	곲	곳	곴	공	곶	곷	곸	곹	곺	곻	과	곽	곾	곿
		0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F

한글 UNICODE 생성/분리 규칙

❑ 한글을 UNICODE로 변환하는 규칙

- 초성 = {ㄱ, ㄲ, ㄴ, ㄷ, ㄸ, ㄹ, ㅁ, ㅂ, ㅃ, ㅅ, ㅆ, ㅇ, ㅈ, ㅉ, ㅊ, ㅋ, ㅌ, ㅍ, ㅎ} : 19 개
- 중성 = {ㅏ, ㅐ, ㅑ, ㅒ, ㅓ, ㅔ, ㅕ, ㅖ, ㅗ, ㅘ, ㅙ, ㅚ, ㅛ, ㅜ, ㅝ, ㅞ, ㅟ, ㅠ, ㅡ, ㅢ, ㅣ} : 21개
- 종성 = { 없음, ㄱ, ㄲ, ㄳ, ㄴ, ㄵ, ㄶ, ㄷ, ㄹ, ㄺ, ㄻ, ㄼ, ㄽ, ㄾ, ㄿ, ㅀ, ㅁ, ㅂ, ㅄ, ㅅ, ㅆ, ㅇ, ㅈ, ㅊ, ㅋ, ㅌ, ㅍ, ㅎ} : 28개
- UNICODE(초,중,종) = 초성x(중성갯수21x종성갯수28) + 중성x종성갯수28 + 종성 + AC00

❑ 예제)

- '가' = [0 x 21 + 0]x28 + 0 + AC00 = 00+AC00 (44032)
- '곽' = ['ㄱ'0 x 21 + 'ㅘ'9]x28 + 'ㄱ'1 + AC = 253 + AC00 = FD + AC00 = ACFD

❑ UNICODE에서 초중종성 분리하는 방법

- 초성 = (U – AC00) / (21 * 28), 중성 = [(U – AC00) % (21 * 28)]/28, 종성 = (U - AC00)% (21*28)

번호	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28
초성	ㄱ	ㄲ	ㄴ	ㄷ	ㄸ	ㄹ	ㅁ	ㅂ	ㅃ	ㅅ	ㅆ	ㅇ	ㅈ	ㅉ	ㅊ	ㅋ	ㅌ	ㅍ	ㅎ										
중성	ㅏ	ㅐ	ㅑ	ㅒ	ㅓ	ㅔ	ㅕ	ㅖ	ㅗ	ㅘ	ㅙ	ㅚ	ㅛ	ㅜ	ㅝ	ㅞ	ㅟ												
종성	없음	ㄱ	ㄲ	ㄳ	ㄴ	ㄵ	ㄶ	ㄷ	ㄹ	ㄺ	ㄻ	ㄼ	ㄽ	ㄾ	ㄿ	ㅀ	ㅁ	ㅂ	ㅄ	ㅅ	ㅆ	ㅇ	ㅈ	ㅊ	ㅋ	ㅌ	ㅍ	ㅎ	

UTF-8 전송 코딩 규칙

❑ 전송 규칙

- 1바이트는 0으로 시작. 2바이트는 11로 시작, 3바이트는 111로 시작, 4바이트는 1111로 시작
- 따라오는 바이트는 10으로 시작

❑ 예) U+C704

- UTF-8 : 코드범위가 010000-10FFFF 범위이므로 3바이트 1110xxxx 10xxxxxx 10xxxxxxxxxx 로 코딩된다.
- C704 = 1100_0111_0000_0100 → XXXX 위치에 순서대로 대입
- 1110xxxx 10xxxxxx 10xxxxxxxxxx = 11101100 10011100 10000100

16비트 코드 범위	UTF-16BE 표현(이진법)	UTF-8 표현(이진법)	설명
000000-00007F	00000000 0xxxxxxx	0xxxxxxx	ASCII와 동일한 범위
000080-0007FF	00000xxx xxxxxxxx	110xxxxx 10xxxxxx	첫 바이트는 110 또는 1110으로 시작하고, 나머지 바이트들은 10으로 시작함
000800-00FFFF	**xxxxxxxx xxxxxxxx**	**1110xxxx 10xxxxxx 10xxxxxx**	
010000-10FFFF	110110ZZ ZZxxxxxx 1101 11xx xxxxxxxx	11110zzz 10zzxxxx 10xxxxxx 10xxxxxx	UTF-16 서러게이트 쌍 영역 (ZZZZ = zzzzz - 1). UTF-8로 표시된 비트 패턴은 실제 코드 포인트와 동일하다. 주1)

주1) 32비트 유티코드 서러게이트에 대한 것은 https://ko.Wikipedia.org/wiki/UTF-16 참조하라.

Thank you

Chapter 03 디지털 코드